

Complaint Intelligence Engine

Extension Modules: What to Build, Why, and How

M.Sc. Data Science — NMIMS NSoMASA Mumbai

Purpose:

This document defines six production-grade extensions to the base Complaint Intelligence Engine project. Each module addresses a real problem that Indian e-commerce companies (Myntra, Meesho, Flipkart, Nykaa, Amazon India) actively invest in solving. Every library and model referenced is 100% free and open-source.

All 6 modules together transform the project from a complaint discovery system into a full decision support platform covering: severity triage, automated routing, churn prediction, competitive benchmarking, response generation, and topic drift detection.

1. Why Extend? The Gap Between Discovery and Decision

The base Complaint Intelligence Engine discovers and displays complaints. It answers: 'What are people complaining about, and which complaints are anomalous?' That is valuable — but it is a passive system. The company reads the dashboard and then has to decide what to do manually.

Real data science products for e-commerce answer a different set of questions: How severe is this complaint? Who should own it? Which user is about to churn? Are we worse than our competitors? What should we say in response? Are the complaints changing in nature?

The six modules in this document answer exactly those six questions. Each module builds on the base project's outputs (cluster labels, SBERT embeddings, anomaly scores, temporal data) — nothing is rebuilt from scratch. The base project is the foundation; these modules are the application layer on top of it.

1.1 How the Modules Connect

The modules form a layered decision pipeline, not six independent additions:

Module	Takes input from	Feeds output to
01 — Severity Predictor	SBERT embeddings + engineered features	02 (high-severity complaints routed first), 04 (health score includes severity)
02 — Auto-Router	Cluster labels (from base) + SBERT embeddings	03 (routing category is a churn feature), 05 (routing category selects response template)
03 — Churn Predictor	Cluster labels, severity scores, review trajectories	04 (churn rate is part of competitive health score)
04 — Benchmarker	Severity scores, cluster labels, churn signals, temporal data	Standalone — the final synthesis module
05 — Auto-Responder	Routing category (from 02) + complaint text	Standalone — requires routing as prerequisite
06 — Drift Detector	Raw text + SBERT embeddings + temporal data	04 (drift events are signals in competitive health score)

2. Final Streamlit Application Architecture

With all six modules integrated, the Streamlit app grows from 4 pages to 10 pages organised across three navigation groups. All pages share a global sidebar with a platform filter, date range selector, and a Refresh button that re-runs the full live ingestion pipeline.

Page	Module	What it shows
1. Live Pulse	Base	KPI cards: total reviews, platforms, critical alerts, top cluster today. Refresh button.
2. Complaint Landscape	Base	Interactive t-SNE scatter coloured by cluster. Hover = full review text. Platform + date filters.
3. Spike Tracker	Base	Weekly complaint volume per cluster per platform. Spike weeks as red markers.
4. Critical Alerts	Base	Tier 1 Critical reviews ranked by anomaly score with severity badges.
5. Severity Triage	Module 01	Incoming reviews sorted by severity score. Alert threshold slider. SHAP word cloud.
6. Complaint Router	Module 02	Paste review → routing category + confidence + SHAP word highlights. Routing load dashboard.
7. Churn Risk	Module 03	User churn risk leaderboard. Kaplan-Meier curve. Complaint pattern explorer.
8. Competitive Intel	Module 04	Radar chart. Health Score timeline. Statistical significance matrix. 4-week forecast.
9. Auto-Responder	Module 05	Paste complaint → 3 generated responses with quality scores. Model comparison panel.
10. Drift Monitor	Module 06	Topic river chart. Drift alert timeline. 'What changed this week' panel.

3. Extension Modules — Full Specifications

01

Complaint Severity Predictor

Will this complaint escalate to a refund demand, legal complaint, or viral social post?

Why build this — the real-world problem

Indian e-commerce platforms receive tens of thousands of reviews every day. Not all complaints are equal — a 1-star review saying "delivery was slightly late" is categorically different from "I was charged twice, my account is blocked, and I am filing a consumer court case." Without an automated severity scoring system, operations teams treat all complaints identically, leading to missed escalations, delayed responses on high-risk cases, and wasted effort on low-priority ones. This module builds a severity predictor that assigns every incoming review a continuous risk score from 0 to 1, and a 3-class severity label (Low / Medium / High). It uses programmatic weak labelling — a production technique pioneered by Stanford NLP and used by companies like Snorkel AI, Flipkart Trust & Safety, and Amazon CX — to generate training labels from heuristics rather than manual annotation, making this fully zero-cost. The business case: a platform that can automatically surface the top 2% of complaints for immediate human review reduces response time on critical issues from days to hours, directly preventing consumer court filings and viral complaint posts.

Implementation steps

1

Build a weak labelling function to create noisy training labels. Rules: reviews with keywords 'fraud', 'legal', 'police', 'consumer court', 'NCDRC', 'chargeback', 'harassment' = High. Reviews with 1-2 stars AND thumbs_up > 20 = Medium-High. All others = Low. Apply to cleaned_reviews.csv to generate a severity_label column.

2

This is programmatic labelling — no manual annotation needed. The labels are noisy but sufficient for training a good classifier, especially with ensemble methods that are robust to label noise.

3

Feature engineering for the classifier: use your existing SBERT embeddings (384-dim) as primary features. Additionally engineer: review_length, star_rating, thumbs_up_log, has_legal_keyword (binary), has_money_keyword (binary), exclamation_count, capital_ratio.

4

Train 5 classifiers on 80% of the weakly-labelled data. Evaluate on the 20% holdout. Report weighted F1 (not accuracy — classes are imbalanced) and calibration curves for each model.

5

Select the best model based on weighted F1 on the validation set. Wrap it as a real-time scorer: every new review that enters the pipeline gets scored before any other processing step.

6

Cross-tabulate severity scores by platform. Which platform has the highest proportion of High-severity complaints? This is a directly actionable business insight.

Models to train and compare

All models below are 100% free and open-source. Train all of them, evaluate on the same test set, and select the best based on the primary metric stated in the evaluation section. The comparison table is a required deliverable — do not just run one model.

Model / Algorithm	Why include it	Free & open-source?
Logistic Regression	Baseline. Interpretable coefficients reveal which features (keywords, star rating, thumbs_up) drive severity. Fast to train on 2,700 samples.	Yes
Random Forest	Robust to noisy weak labels due to ensemble averaging. Feature importance ranking shows the relative weight of embedding features vs engineered features.	Yes
XGBoost	Best overall for imbalanced multi-class. Use scale_pos_weight for High severity class (rare). Native SHAP support for explanations.	Yes
LinearSVC	Excellent on high-dimensional TF-IDF features. Fast. Often outperforms deep models on short text classification.	Yes
DistilBERT fine-tuned	distilbert-base-multilingual-cased fine-tuned on your weakly-labelled set. Best accuracy, 40% smaller than BERT. Free via HuggingFace Transformers.	Yes

Evaluation metrics

Evaluate every model on all metrics below. Report results in a table. Justify your final model selection in one paragraph.

Weighted F1 per class	Confusion matrix (normalised)	Calibration curve (reliability diagram)
ROC-AUC per class (one-vs-rest)	Latency per prediction (ms)	Cohen's Kappa vs weak label agreement

Streamlit demo features for this module

>	Incoming review feed sorted by severity score descending — operations team sees highest-risk complaints first
>	Severity heatmap: platform x week x severity level, shows which platforms are trending toward more High-severity complaints
>	Alert threshold slider: user sets a threshold (e.g. score > 0.8) and sees count of reviews above threshold per platform this week
>	SHAP word cloud per severity class: which words are most predictive of High severity specifically
>	Model comparison table showing F1, calibration score, and inference latency for all 5 models with the selected model highlighted

Technology stack for this module

Library	Purpose	Install command	Licence
google-play-scraper	Live review ingestion	pip install google-play-scraper	MIT
scikit-learn	LogReg, RF, LinearSVC, calibration curves	pip install scikit-learn	BSD
xgboost	XGBoost classifier with SHAP	pip install xgboost	Apache 2.0
shap	SHAP explanations for any model	pip install shap	MIT
transformers	DistilBERT fine-tuning	pip install transformers	Apache 2.0
datasets (HuggingFace)	Data loading for fine-tuning	pip install datasets	Apache 2.0
imbalanced-learn	SMOTE oversampling for High severity class	pip install imbalanced-learn	MIT
matplotlib / seaborn	Calibration curves, confusion matrix plots	pip install matplotlib seaborn	BSD / BSD

Why build this — the real-world problem

When Meesho or Flipkart receives 50,000 reviews per month, a human agent decides which complaints go to the logistics team, which to the payment reconciliation team, and which to the product quality team. This triage process is manual, slow, and error-prone — misrouted complaints sit in the wrong queue for days before someone notices. Your clustering in the base project already discovered the natural complaint categories — delivery failure, fake product, payment fraud, app crash, and so on. Module 02 turns those unsupervised discoveries into a supervised routing classifier. This is the exact bridge between unsupervised and supervised learning that examiners and recruiters are impressed by: you used UML to discover structure, then built a supervised model on top of that structure without needing any manual labels. The business case: Freshdesk (used by Myntra) and Zoho Desk charge companies thousands of rupees per month to provide automated complaint routing. This module builds an equivalent capability using entirely free tools and your own data.

Implementation steps

- 1 Use your K-Means cluster labels as training labels for the routing classifier. Only use high-confidence assignments: reviews whose distance to their assigned centroid is below the 40th percentile of all centroid distances. This filters out ambiguous reviews that would confuse the classifier.
- 2 This creates a direct intellectual bridge: your unsupervised work (clustering) generates the labels for your supervised work (routing). Explicitly highlight this in your presentation — it is a methodologically mature design choice.
- 3 Feature options: (a) TF-IDF vectors (5000 features) for traditional ML models, (b) your SBERT embeddings (384-dim) for embedding-based models, (c) a combination of both via feature concatenation. Test all three input representations.
- 4 Add SHAP explanations to every routing prediction. For each routed complaint, show the top 3 words that drove the routing decision. This is what an operations manager needs to trust the model — not just the label, but the reason.
- 5 Train 5 classifiers. Evaluate with macro-F1 (important: complaint categories are imbalanced — delivery failure is common, payment fraud is rare, and macro-F1 weights all classes equally).
- 6 Build the Streamlit page: paste any review text, get routing category + confidence score + highlighted words that drove the prediction. This is the most immediately useful demo page for a recruiter.

Models to train and compare

All models below are 100% free and open-source. Train all of them, evaluate on the same test set, and select the best based on the primary metric stated in the evaluation section. The comparison table is a required deliverable — do not just run one model.

Model / Algorithm	Why include it	Free & open-source?
Multinomial Naive Bayes	Strong baseline on TF-IDF. Fast, interpretable, known to work well on short text. Coefficient = word importance per category.	Yes
SGD Classifier (log loss)	Online learning — can update incrementally as new reviews arrive. Important for a live system. Best for streaming deployment.	Yes
LightGBM	On TF-IDF + engineered features. Fast, strong on multi-class with native categorical support and feature importance.	Yes
SBERT + Softmax head	Add a trainable linear classification layer on top of frozen SBERT embeddings. Transfer learning from your own embedding space.	Yes
Zero-shot (facebook/bart-large-mnli)	Classify complaints without any training using natural language labels. Compare against supervised models to show the value of your labelled data.	Yes

Evaluation metrics

Evaluate every model on all metrics below. Report results in a table. Justify your final model selection in one paragraph.

Macro F1 (primary metric)	Per-class precision and recall	Normalised confusion matrix heatmap
SHAP word importance per routing category	Zero-shot vs supervised accuracy gap	Confidence calibration per category

Streamlit demo features for this module

>	Live text input widget: paste any review → routing category + confidence bar + SHAP highlighted words appear instantly
>	Routing load dashboard: which team is receiving the most complaints this week? Which platform is generating the most work for each team?
>	Misrouting analysis: reviews where the model confidence is below 60% flagged as 'Needs Human Review' — shown separately in the queue
>	Zero-shot vs supervised comparison panel: same reviews scored by both, differences highlighted to show the value of your labelling
>	Model selector: user can switch between all 5 models and see routing change in real time — demonstrates understanding of model differences

Technology stack for this module

Library	Purpose	Install command	Licence
---------	---------	-----------------	---------

scikit-learn	NaiveBayes, SGD, all evaluation metrics	pip install scikit-learn	BSD
lightgbm	LightGBM classifier	pip install lightgbm	MIT
transformers	BART zero-shot, DistilBERT routing head	pip install transformers	Apache 2.0
shap	Word-level SHAP for routing explanations	pip install shap	MIT
sentence-transformers	Reuse SBERT embeddings from base project	pip install sentence-transformers	Apache 2.0
streamlit	Interactive routing demo page	pip install streamlit	Apache 2.0
plotly	Confidence bars, confusion matrix heatmap	pip install plotly	MIT

Why build this — the real-world problem

Churn — users who stop buying — is the most expensive problem in e-commerce. Acquiring a new customer costs 5–7x more than retaining an existing one. The challenge is that companies only know a user has churned after it has already happened: they stopped buying weeks ago, and no one noticed. Reviews are a leading indicator of churn. A user who goes from "great service, always my first choice" (5 stars) to "terrible delivery, never again" (1 star, high severity, delivery failure cluster) is a churning user — and your project captures this signal. This module builds a model that detects churn trajectories before the user actually stops buying, using only the review data you already have. This is one of the top-3 ML problems every e-commerce company actively invests in. Meesho, Myntra, and Flipkart all have internal churn prediction models. Building one from review signals (rather than purchase history, which you don't have) is genuinely novel.

Implementation steps

- 1 Group reviews by platform and user. For users appearing more than once in the corpus, compute a review trajectory: star rating over time, cluster membership over time, anomaly score trend, severity score trend.
- 2 Define the churn proxy label using a combination of signals: user's last review has score ≤ 2 AND has a High severity flag AND has not appeared in the corpus for the most recent 4-week period = churn_signal = 1. All others = 0. This is your binary target.
- 3 Engineer user-level features: last_star_rating, star_delta (last minus first), complaint_cluster_count (how many distinct complaint types), max_anomaly_score, days_since_last_review (if estimable from timestamps), review_frequency (reviews per month), is_hinglish (language of most recent review), platform.
- 4 Since churn is rare (class imbalance), use SMOTE for oversampling the churn class in the training set. Evaluate on an unmodified test set — never apply SMOTE to test data.
- 5 Evaluate using the precision-recall curve rather than ROC-AUC. Precision-recall is the correct metric when classes are imbalanced and false positives are costly (you do not want to alarm a brand about a user who is not actually churning).
- 6 Add Kaplan-Meier survival analysis: what % of users who post a High-severity complaint return to post a positive review within 30 days? This adds a statistical layer that non-ML evaluators find compelling.

Models to train and compare

All models below are 100% free and open-source. Train all of them, evaluate on the same test set, and select the best based on the primary metric stated in the evaluation section. The comparison table is a required deliverable — do not just run one model.

Model / Algorithm**Why include it****Free & open-source?**

Logistic Regression (L1)	Baseline with L1 regularisation for automatic feature selection. Coefficients show which review trajectory features matter most for churn.	Yes
Random Forest	Handles feature interactions well — e.g. high anomaly score AND delivery complaint cluster together = very high churn risk. Built-in feature importance.	Yes
XGBoost with early stopping	Best overall for tabular churn features. Use early stopping on validation loss. SHAP for individual user churn explanation.	Yes
Kaplan-Meier (lifelines)	Survival analysis: estimate the 'survival function' of user engagement post-complaint. Not a classifier — a statistical estimator. Adds methodological depth.	Yes
Isolation Forest (baseline)	Use your existing anomaly scores directly as a churn proxy without training. Compare against supervised models to show the value of feature engineering.	Yes

Evaluation metrics

Evaluate every model on all metrics below. Report results in a table. Justify your final model selection in one paragraph.

Precision-recall curve (primary)	F1 at optimal threshold (Youden's J)	Calibrated probability outputs
SHAP user-level explanation	Kaplan-Meier curve with log-rank test	Lift curve over random baseline

Streamlit demo features for this module

>	Churn risk leaderboard: users ranked by churn probability per platform, showing their review trajectory (star rating trend, last complaint cluster, severity score)
>	Platform churn risk comparison: which app is losing users fastest based on review trajectory signals? Plotted as a weekly trend line
>	Complaint pattern → churn probability explorer: user selects a complaint type and severity level and sees the estimated churn probability for that pattern
>	Kaplan-Meier chart: survival curve showing what % of users who post a High-severity complaint are still active after 1, 2, 4, and 8 weeks
>	SHAP waterfall chart for any individual user: shows exactly which features drove their churn prediction

Technology stack for this module

Library	Purpose	Install command	Licence
---------	---------	-----------------	---------

scikit-learn	LogReg, RF, precision-recall, SMOTE (via imbalanced-learn)	pip install scikit-learn	BSD
xgboost	XGBoost with SHAP	pip install xgboost	Apache 2.0
shap	User-level SHAP waterfall charts	pip install shap	MIT
lifelines	Kaplan-Meier survival analysis, log-rank test	pip install lifelines	MIT
imbalanced-learn	SMOTE oversampling for churn class	pip install imbalanced-learn	MIT
plotly	KM curves, precision-recall plots, leaderboard table	pip install plotly	MIT

Competitive Intelligence Benchmarker

Objectively: which platform is winning and losing on each complaint dimension, with statistical proof?

Why build this — the real-world problem

Every platform's product and strategy team wants to know: are we worse than Meesho on delivery complaints? Is Nykaa's fake product problem improving or worsening month-on-month compared to Myntra? No public benchmark for this exists. Consulting firms charge companies crores to produce competitive intelligence reports that answer exactly these questions. Your project already has multi-platform complaint data with cluster labels, severity scores, and temporal information. This module uses that data to build an objective, statistically validated competitive benchmarking system. The key word is 'statistically validated' — you will not just rank platforms, you will prove that the differences are statistically significant using non-parametric tests, which is what separates a rigorous analysis from a dashboard that someone made up. This is also the module that will impress consulting firm recruiters (McKinsey, BCG, Bain, Deloitte) the most — it speaks their language: benchmarks, significance, and trend analysis.

Implementation steps

- 1 For each platform × complaint cluster combination, compute: complaint rate (complaints / total reviews for that platform), median severity score, anomaly rate (% flagged as critical), Tier 1 Critical density, and week-over-week complaint volume trend.
- 2 Build a Composite Health Score per platform per complaint dimension: normalised 0-100 composite of (1 - complaint_rate), (1 - median_severity), (1 - anomaly_rate). Higher score = healthier platform on that dimension. Document the weighting formula so it is reproducible.
- 3 Statistical validation using Mann-Whitney U test: for each pair of platforms and each complaint cluster, test whether the severity score distributions are significantly different. Report p-value and effect size (Cohen's d). This gives you statements like: 'Meesho is significantly worse than Flipkart on delivery complaints ($p < 0.01$, $d = 0.44$).'
- 4 ANOVA + Tukey HSD for simultaneous comparison: compare all 5 platforms on each complaint dimension at once with pairwise correction for multiple comparisons. This prevents false discoveries from running too many individual t-tests.
- 5 Trend analysis: fit a linear regression on weekly Health Scores for each platform per complaint cluster. Slope = rate of improvement. p-value = is the trend statistically real? A platform with a significantly negative slope is deteriorating — a key business risk signal.
- 6 Forecast layer using ARIMA: for each platform × cluster time series, fit ARIMA (auto order selection) to forecast the next 4 weeks of complaint volume. This makes the benchmarker forward-looking.

Models to train and compare

All models below are 100% free and open-source. Train all of them, evaluate on the same test set, and select the best based on the primary metric stated in the evaluation section. The comparison table is a required deliverable — do not just run one model.

Model / Algorithm	Why include it	Free & open-source?
Mann-Whitney U test	Non-parametric test for comparing two severity distributions. No normality assumption — appropriate for skewed review data.	Yes
ANOVA + Tukey HSD	Compare all platforms simultaneously with pairwise corrections. Prevents false discoveries from multiple comparisons.	Yes
Linear regression (trend)	Fit a trend line on weekly Health Scores. Slope and p-value tell you if a platform is improving or deteriorating.	Yes
ARIMA (auto_arima)	Forecast next-4-week complaint volume per platform per cluster. Forward-looking competitive intelligence.	Yes
Bootstrap confidence intervals	Compute 95% CI on all Health Scores by resampling. Shows which platform rankings are statistically reliable vs noisy.	Yes

Evaluation metrics

Evaluate every model on all metrics below. Report results in a table. Justify your final model selection in one paragraph.

Mann-Whitney U p-values (significance matrix)	Cohen's d effect sizes for each comparison	Trend regression R-squared and slope p-value
ARIMA forecast MAPE	Bootstrap CI width on Health Scores	Correlation between Health Score and star rating (external validation)

Streamlit demo features for this module

>	Radar chart: all 5 platforms across 6 complaint dimensions simultaneously — the visual centrepiece of this module
>	Health Score timeline: weekly trend lines per platform with confidence bands, showing which platforms are improving and which are deteriorating
>	Statistical significance matrix: a heatmap showing p-values for all platform pairs on each complaint dimension, with cells coloured by significance level
>	Forecast panel: next-4-week complaint volume forecast per platform with prediction intervals — which platforms are expected to get worse?
>	Auto-generated insight text: 'Meesho is significantly worse than all other platforms on delivery complaints ($p < 0.01$). Its Health Score has declined -2.3 points per week over the last 8 weeks.'

Technology stack for this module

Library	Purpose	Install command	Licence
scipy.stats	Mann-Whitney U, ANOVA, Shapiro-Wilk normality test	pip install scipy	BSD
statsmodels	Tukey HSD, OLS regression for trend analysis	pip install statsmodels	BSD
pmdarima	auto_arima for ARIMA order selection and forecasting	pip install pmdarima	MIT
plotly	Radar chart, trend lines with CI bands, heatmap	pip install plotly	MIT
numpy	Bootstrap resampling for confidence intervals	pip install numpy	BSD
pandas	Weekly groupby aggregations for time series	pip install pandas	BSD

Auto-Response Generator with Quality Scoring

Generate platform-appropriate responses to complaints and score them on empathy, specificity, and actionability

Why build this — the real-world problem

Indian e-commerce platforms employ hundreds of customer service agents to respond to Google Play reviews. The typical response is a generic, copy-pasted template: "We are sorry for your experience. Please contact support at help@platform.com." This response does nothing — it does not address the complaint, does not offer resolution, and often makes the user angrier. This module uses your complaint classification (from Module 02) to generate contextually appropriate, category-specific responses using a free, locally-runnable language model. It then scores each generated response on three quality dimensions — empathy, specificity, and actionability — using a lightweight regression model trained on a small set of manually scored examples. Companies like Freshdesk and Zoho Desk sell this as a premium feature for thousands of rupees per month. You will build it using entirely free, open-source tools. Mistral-7B-Instruct runs on CPU (slowly but functionally) and is free under Apache 2.0.

Implementation steps

- 1 For each complaint category from Module 02's routing classifier, craft a structured prompt template. The template includes: platform name, complaint cluster label, extracted complaint summary (first 200 chars), tone instruction (formal/empathetic/resolution-focused), and a few-shot example of a good response for that category.
- 2 Use Mistral-7B-Instruct via the HuggingFace transformers library or the free Ollama local inference server. Generate 3 response variants per complaint: Tone A (Formal Apology), Tone B (Empathetic Listening), Tone C (Resolution Focused). All free, runs on CPU.
- 3 Build a Response Quality Scorer: manually score 30 generated responses across three dimensions — Empathy (0-3), Specificity (0-3, does it mention the specific complaint?), Actionability (0-3, does it offer a concrete resolution step?). Train a Ridge regression on features extracted from the response text: response_length, contains_apology, mentions_specific_issue (TF-IDF overlap with complaint), no_generic_phrases_count, BERTScore precision against a reference response.
- 4 BERTScore is a free metric that measures semantic similarity between a generated response and a reference response using BERT embeddings. It correlates well with human quality judgements and requires no manual labelling beyond your initial 30 examples.
- 5 Evaluate all three generated variants using the quality scorer. Select the highest scoring variant automatically. Track which tone type consistently generates the highest quality responses per complaint category.
- 6 Compare Mistral-7B-Instruct quality against a rule-based template baseline (the current industry standard). Show that the model outperforms the template on specificity and actionability even with just a few-shot prompt.

Models to train and compare

All models below are 100% free and open-source. Train all of them, evaluate on the same test set, and select the best based on the primary metric stated in the evaluation section. The comparison table is a required deliverable — do not just run one model.

Model / Algorithm	Why include it	Free & open-source?
Mistral-7B-Instruct v0.3	Best free open-source instruction-following model. Runs via Ollama (free local inference) or HuggingFace pipeline. Apache 2.0 licence.	Yes
Phi-3-mini-4k-instruct	Microsoft's 3.8B parameter model — faster than Mistral-7B on CPU, still good quality. Free via HuggingFace.	Yes
BERTScore	Automatic reference-free quality metric using BERT embeddings. Measures precision, recall, F1 between generated and reference responses.	Yes
Ridge Regression (quality scorer)	Lightweight quality prediction model trained on 30 manually-scored responses. Fast, interpretable, only needs a tiny labelled set.	Yes
Template baseline	Rule-based response using complaint category → template mapping. The current industry standard. Exists to show how much the LLM improves over it.	Yes

Evaluation metrics

Evaluate every model on all metrics below. Report results in a table. Justify your final model selection in one paragraph.

BERTScore (P, R, F1) vs reference responses	Quality regressor score (empathy + specificity + actionability)	Human evaluation on 20 sampled responses (spot check)
Response latency per model (CPU vs GPU)	Generic phrase rate (lower = more specific, better)	LLM vs template quality delta

Streamlit demo features for this module

>	Live complaint input → 3 response variants generated with quality scores for each — user selects the best one or edits it before sending
>	Quality score breakdown per response: empathy bar, specificity bar, actionability bar, with colour coding
>	Model comparison panel: Mistral-7B vs Phi-3-mini vs template baseline on the same complaint — quality scores and latency compared side by side
>	Response history log: all generated responses with their scores, exportable as CSV for the platform's customer service team
>	Tone performance analysis: which tone type (formal / empathetic / resolution) consistently scores highest for each complaint category?

Technology stack for this module

Library	Purpose	Install command	Licence
transformers (HuggingFace)	Load Mistral-7B-Instruct or Phi-3-mini for inference	pip install transformers	Apache 2.0
ollama	Free local inference server for running Mistral or Phi-3 without writing model code	ollama.com (free download)	MIT
bert-score	BERTScore automatic evaluation metric	pip install bert-score	MIT
scikit-learn	Ridge regression quality scorer, TF-IDF for specificity feature	pip install scikit-learn	BSD
accelerate	HuggingFace accelerate for efficient CPU/GPU inference	pip install accelerate	Apache 2.0
streamlit	Interactive response generation and comparison UI	pip install streamlit	Apache 2.0
pandas	Response history storage and CSV export	pip install pandas	BSD

Why build this — the real-world problem

Volume spikes are easy to detect. Topic drift is harder and more dangerous. Imagine Meesho's complaint volume stays constant, but the nature of delivery complaints changes from "late delivery" to "wrong item delivered" to "item damaged in transit" — each shift reflecting a different logistics failure requiring a different operational fix. If you only monitor volume, you miss all three. Topic drift detection identifies when the thematic structure of complaints is shifting — before it compounds into a crisis. This is a research-level technique that is beginning to appear in production at companies like Amazon's Voice of Customer team and Flipkart's NLP research group. Building it from scratch using free tools at the student level is genuinely impressive. This module also adds the most methodologically novel contribution to your project: ADWIN (Adaptive Windowing), a streaming concept drift detection algorithm that can operate on your SBERT embedding space in real time.

Implementation steps

- 1 Split your corpus into 4-week time windows. For each window, compute two representations: (a) a TF-IDF document-term matrix for LDA topic modelling, and (b) the mean SBERT embedding vector for the window for embedding-space drift detection.
- 2 Apply LDA (Gensim) separately to each time window. Extract the top-10 words per topic per window. To compare topics across windows, compute cosine similarity between topic-word probability vectors in adjacent windows. Low cosine similarity = the topic changed.
- 3 Apply BERTopic to each time window as a comparison to LDA. BERTopic uses your SBERT embeddings under the hood, so it integrates naturally with your existing pipeline. Compare topic coherence scores (C_v metric) between LDA and BERTopic.
- 4 Statistical drift test: compute Jensen-Shannon Divergence (JSD) between the TF-IDF term distributions of consecutive windows. JSD ranges from 0 (identical) to 1 (completely different). Use a permutation test to establish a significance threshold for 'real' drift vs noise-driven divergence.
- 5 ADWIN on embedding space: use the River library's ADWIN implementation. Feed it the weekly mean SBERT embedding norms. ADWIN will automatically detect when the statistical distribution of embeddings shifts beyond what random variation can explain. This is your most technically sophisticated component.
- 6 For each detected drift event, generate an automatic description: extract the TF-IDF terms that appear in the new window but not the previous one. These 'emerging terms' are what the drift is about. Display them as 'What changed this week' in the dashboard.

Models to train and compare

All models below are 100% free and open-source. Train all of them, evaluate on the same test set, and select the best based on the primary metric stated in the evaluation section. The comparison table is a required deliverable — do not just run one model.

Model / Algorithm	Why include it	Free & open-source?
LDA (Gensim)	Interpretable topic models per time window. Topic-word distributions are easy to compare across windows using cosine similarity.	Yes
BERTopic	SBERT-powered topic modelling. Better coherence than LDA. Integrates with your existing embedding pipeline.	Yes
Jensen-Shannon Divergence	Information-theoretic drift measure between consecutive window TF-IDF distributions. No model training required.	Yes
ADWIN (River library)	Streaming concept drift detector that adapts its window size automatically. Detects when SBERT embedding distribution shifts significantly.	Yes
NMF (scikit-learn)	Non-negative Matrix Factorisation as a third topic modelling approach. Compare topic coherence (C_v) across LDA, BERTopic, and NMF.	Yes

Evaluation metrics

Evaluate every model on all metrics below. Report results in a table. Justify your final model selection in one paragraph.

Topic coherence score C_v (higher = more coherent topics)	Jensen-Shannon Divergence between consecutive windows	ADWIN drift detection latency (how quickly does it detect a real drift?)
Manual topic validity check (spot-read 5 topics per window)	Recall of drift events against known platform incidents (e.g. known outage dates)	

Streamlit demo features for this module

>	Topic evolution river chart (Plotly Sankey diagram): shows how each topic's share of complaint volume changes week by week, with streams widening or narrowing as topics grow or shrink
>	Drift alert timeline: flagged drift events shown as markers on the complaint volume chart, with a tooltip showing before/after topic comparison
>	'What changed this week' auto-summary panel: emerging terms highlighted in green, disappearing terms in red, with the JSD score and ADWIN signal
>	LDA vs BERTopic vs NMF side-by-side topic comparison panel: same time window, different models, coherence scores displayed
>	Platform-specific drift view: select a platform and see its topic drift history — which platforms have the most volatile complaint profiles?

Technology stack for this module

Library	Purpose	Install command	Licence
gensim	LDA topic modelling, topic coherence C_v metric	pip install gensim	LGPL
bertopic	BERTopic with SBERT integration	pip install bertopic	MIT
river	ADWIN streaming concept drift detection	pip install river	BSD
scipy.spatial	Jensen-Shannon Divergence computation	pip install scipy	BSD
scikit-learn	NMF topic modelling	pip install scikit-learn	BSD
plotly	Sankey river chart, drift timeline, topic comparison panels	pip install plotly	MIT
pyldavis	Interactive LDA topic visualisation for notebooks	pip install pyldavis	BSD

4. Complete Technology Stack — All Modules

Every library below is free, open-source, and installable via pip (except Ollama which is a free local download). No paid APIs, no cloud credits, and no commercial licences are required for any part of this project.

Library	Category	Purpose	Install command	Licence	Modules
google-play-scraper	Data ingestion	Live reviews from Play Store, no auth	pip install google-play-scraper	MIT	Base + M01
praw	Data ingestion	Reddit posts via API	pip install praw	BSD	Base
pandas	Data processing	All dataframe operations	pip install pandas	BSD	All
numpy	Numerical computing	Embedding arrays, bootstrapping	pip install numpy	BSD	All
scipy	Statistical tests	MWU, ANOVA, JSD, bootstrap	pip install scipy	BSD	M04 + M06
langdetect	Language detection	Classify en/hi/mixed reviews	pip install langdetect	Apache 2.0	Base
scikit-learn	ML framework	All classical ML models, metrics, PCA, TF-IDF	pip install scikit-learn	BSD	All
sentence-transformers	NLP embeddings	Multilingual SBERT (paraphrase-multilingual-MiniLM-L12-v2)	pip install sentence-transformers	Apache 2.0	Base
umap-learn	Dimensionality reduction	UMAP 10-dim for clustering	pip install umap-learn	BSD	Base
hdbscan	Density clustering	HDBSCAN + noise point detection	pip install hdbscan	BSD	Base
xgboost	Gradient boosting	Severity, routing, churn prediction	pip install xgboost	Apache 2.0	M01+M02+M03
lightgbm	Gradient boosting	Fast multi-class routing classifier	pip install lightgbm	MIT	M02
shap	Explainability	SHAP values for all tree models	pip install shap	MIT	M01+M02+M03
imbalanced-learn	Class balancing	SMOTE for severity and churn classes	pip install imbalanced-learn	MIT	M01+M03

lifelines	Survival analysis	Kaplan-Meier curves, log-rank test	pip install lifelines	MIT	M03
statsmodels	Statistical modelling	Tukey HSD, OLS trend regression	pip install statsmodels	BSD	M04
pmdarima	Time series	auto_arima forecast for complaint volume	pip install pmdarima	MIT	M04
transformers	LLM inference	Mistral-7B, Phi-3-mini, DistilBERT fine-tuning	pip install transformers	Apache 2.0	M01+M05
accelerate	Efficient inference	HuggingFace accelerate for CPU/GPU	pip install accelerate	Apache 2.0	M05
bert-score	Response quality	Automatic BERTScore evaluation	pip install bert-score	MIT	M05
gensim	Topic modelling	LDA, topic coherence C_v metric	pip install gensim	LGPL	M06
bertopic	Topic modelling	BERTopic with SBERT integration	pip install bertopic	MIT	M06
river	Concept drift	ADWIN streaming drift detection	pip install river	BSD	M06
plotly	Visualisation	All interactive charts in Streamlit	pip install plotly	MIT	All
streamlit	Dashboard	7-page live Streamlit app	pip install streamlit	Apache 2.0	All
joblib	Model persistence	Save/load all trained models	pip install joblib	BSD	All
matplotlib / seaborn	Static plots	Confusion matrices, calibration curves	pip install matplotlib seaborn	BSD	All
pyldavis	LDA visualisation	Interactive topic explorer in notebooks	pip install pyldavis	BSD	M06
ollama	Local LLM inference	Run Mistral-7B locally — free, no API needed	ollama.com (free download)	MIT	M05

datasets (HuggingFace)	Data loading	Dataset formatting for fine-tuning	pip install datasets	Apache 2.0	M01+M05
---------------------------	--------------	--	-------------------------	---------------	---------

Note on Ollama: Ollama (ollama.com) is a free, open-source tool for running large language models locally on your laptop. It provides one-command installation of Mistral-7B and Phi-3-mini. No API key, no cloud account, no usage cost. Install with: `curl -fsSL https://ollama.com/install.sh | sh` (Linux/Mac) or download from ollama.com (Windows).

5. Recommended Build Order and Rationale

The modules are not independent. Build them in the order below so that each module's outputs are available as inputs to the next.

Order	Module	Why this sequence
1st	02 — Auto-Router	Directly uses existing cluster labels. Very buildable in 1-2 days. Gives you a clear supervised task immediately after finishing clustering.
2nd	01 — Severity Predictor	Weak labelling is fast (rule-based). The severity score enriches every subsequent module. Do this before churn and benchmarking.
3rd	04 — Competitive Benchmarker	Mostly statistics on data you already have. No new ML model training. High output value for low implementation effort.
4th	03 — Churn Predictor	Requires severity scores (from 01) and routing labels (from 02) as features. User-level feature engineering takes time — do it after the easier modules.
5th	06 — Drift Detector	Technically the most novel. Requires complete temporal data from the base project. ADWIN setup is non-trivial — allocate extra time.
6th	05 — Auto-Responder	Involves LLM inference setup (Ollama or HuggingFace pipeline). Do last so it does not block other modules if setup takes time.

6. Anticipated Examiner Questions — All Modules

Q1: How do you justify using weak labels for training the severity classifier?

Programmatic weak labelling is a well-established technique in production ML (Snorkel AI, 2019; Ratner et al.). It trades label accuracy for scale — you can label thousands of examples using heuristics rather than hundreds by hand. Our weak labels are based on star rating + thumbs_up + keyword matching, which have high precision for High and Low severity even if recall is imperfect. We trained on weak labels and evaluated on 50 manually verified examples to confirm that the classifier generalises beyond the labelling heuristics.

Q2: Why use macro-F1 for the routing classifier instead of accuracy?

Complaint categories are imbalanced — delivery failure complaints are far more common than payment fraud complaints. Accuracy would be dominated by the majority class. Macro-F1 computes F1 separately for each class and averages them, giving equal weight to rare categories like payment fraud. Since misrouting a payment fraud complaint is far more costly than misrouting a delivery complaint, we want the model penalised equally for errors on all categories.

Q3: What is SMOTE and why do you use it for churn prediction?

SMOTE (Synthetic Minority Oversampling Technique) generates synthetic samples of the minority class (churn = 1) by interpolating between existing minority examples in feature space. It addresses class imbalance during training without duplicating examples, which would cause overfitting. Critically, SMOTE is only applied to the training set — the test set remains unmodified so evaluation reflects real-world class proportions.

Q4: What does Jensen-Shannon Divergence measure in the topic drift module?

JSD measures how different two probability distributions are. In our context, each time window's complaint corpus produces a TF-IDF term distribution — the probability of each word appearing in that window's complaints. JSD between consecutive windows measures how much the vocabulary of complaints has changed. A JSD near 0 means complaints are using the same words (topics are stable); JSD near 1 means complaints in the new window use completely different vocabulary (topics have drifted).

Q5: Why use ADWIN instead of a simple rolling mean for drift detection?

A rolling mean requires you to choose a fixed window size in advance. ADWIN (Adaptive Windowing) automatically adjusts its window size based on the data: it maintains a large window when the distribution is stable (more data = more statistical power) and shrinks the window when it detects a shift (focusing on recent data only). This makes it adaptive to both gradual drift and sudden changes, which are the two most common types of complaint topic shift.

Q6: How do you evaluate the quality of generated responses without extensive human labels?

We use BERTScore, an automated metric that measures the semantic similarity between a generated response and a reference response using contextualised BERT embeddings. BERTScore has been shown to correlate strongly with human quality judgements on dialogue tasks (Zhang et al., 2020). We supplement this with a lightweight Ridge regression quality scorer trained on 30 manually rated responses — sufficient for a regression on 5 features. The combination gives us both automated scale and human-grounded calibration.

7. Updated Resume Bullet — Full Project

Use this bullet after completing all six modules. It replaces the earlier resume bullet.

Full version (detailed):

Built Complaint Intelligence Engine for Indian e-commerce: live ingestion of 2,500+ reviews from Myntra, Meesho, Nykaa, Flipkart, and Amazon India via Google Play API + Reddit PRAW; multilingual Sentence-BERT (384-dim) embeddings with Hinglish support; K-Means + HDBSCAN complaint archetype discovery; PCA + UMAP + t-SNE pipeline. Extended with 6 production modules: (1) XGBoost/DistilBERT severity predictor using programmatic weak labelling; (2) LightGBM + SHAP complaint auto-router from unsupervised cluster labels; (3) XGBoost churn signal predictor with Kaplan-Meier survival analysis; (4) statistically validated competitive benchmarker using Mann-Whitney U, Tukey HSD, ARIMA forecasting; (5) Mistral-7B auto-response generator with BERTScore quality evaluation; (6) LDA + BERTopic topic drift detector with ADWIN streaming concept drift detection. Deployed as 10-page live Streamlit dashboard.

Short version (for space-constrained resume):

Complaint Intelligence Engine | Python, Sentence-BERT, HDBSCAN, XGBoost, Mistral-7B, Streamlit — 7-stage unsupervised NLP pipeline on live Indian e-commerce reviews; extended with severity prediction, complaint auto-routing (SHAP-explained), churn signal detection (Kaplan-Meier), competitive benchmarking (Mann-Whitney/ARIMA), LLM-based response generation (BERTScore), and topic drift detection (ADWIN). 10-page live dashboard.